

AI Engineer in 1 Year

A Complete Step-by-Step Roadmap

Months 1-3 -- Foundations



Months 4-6 -- Core ML



Months 7-9 -- Specialize



Months 10-12 -- Ship It



Designed for beginners | Python-starting | Project-first approach

Generated: July 30, 2026

Table of Contents

- 1 The Big Picture -- What Is an AI Engineer?
- 2 Months 1-3: Foundations (Power-Up Phase)
 - 2.1 Python You Already Need
 - 2.2 The Tool Stack -- numpy, pandas, matplotlib
 - 2.3 Just-Enough Math for AI
 - 2.4 Mini-Project: Linear Regression from Scratch
- 3 Months 4-6: Core Machine Learning
 - 3.1 Classical ML with scikit-learn
 - 3.2 Deep Learning Foundations with PyTorch
 - 3.3 MLOps Basics
 - 3.4 Mini-Project: Cat vs Dog Classifier
- 4 Months 7-9: Specialize & Level Up
 - 4.1 Track A -- NLP / LLMs (Recommended)
 - 4.2 Track B -- Computer Vision
 - 4.3 Track C -- ML Platform / Infra
- 5 Months 10-12: Production & Portfolio
 - 5.1 Docker, CI/CD, Model Serving
 - 5.2 Monitoring & Evaluation
 - 5.3 Portfolio Projects That Stand Out
- 6 Resources & Links
- 7 Weekly Action Plan
- 8 The Secret Sauce

1. The Big Picture -- What Is an AI Engineer?

An AI engineer is someone who teaches computers to learn on their own. Instead of writing step-by-step instructions ("if the button is red, click it"), you feed the computer thousands of examples and let it figure out the pattern itself.

Think of it like teaching a child to recognise cats: you don't explain whiskers and fur patterns -- you just show them lots of cats (and non-cats) until they get it.

What you will be able to do in 12 months

- Write Python that cleans, explores, and visualises real-world data.
- Train a model to predict prices, classify images, or understand text.
- Deploy that model so other people (or apps) can use it over the web.
- Know when AI is the wrong tool -- and reach for a simple rule instead.
- Read a research paper and understand enough to try the idea yourself.
- Debug a training pipeline when the loss isn't going down.

The Mindset Shift

Normal code is deterministic: same input = same output. AI code is statistical: you give it examples, it finds patterns, and it gets better over time. This changes everything about how you debug, test, and trust your code.

2. Months 1-3 -- Foundations (Power-Up Phase)

You already know Python basics and how to write tests. Now we layer on the specific tools and concepts you will use every single day as an AI engineer.

2.1 Python You Already Need

Beyond basic Python, make sure you are comfortable with these -- they show up constantly in AI code:

- [x] List/dict comprehensions -- cleaner than raw loops
- [x] Generator expressions -- memory-efficient iteration over huge datasets
- [x] lambda, map, filter, functools.partial
- [x] Decorators -- used by Flask, PyTorch, and many ML frameworks
- [x] Context managers (with open(...) as f:)
- [x] *args' and '**kwargs' -- you will see these in every library
- [x] Type hints -- you already use them; they keep ML code maintainable
- [x] Virtual environments (venv / conda) -- isolate your project dependencies

2.2 The Tool Stack -- numpy, pandas, matplotlib

These three libraries are the Python data stack. Learn them early because every ML library sits on top of them.

numpy

NumPy gives you arrays that are 10-100x faster than Python lists. You will use it for all math: matrix multiplication, random numbers, statistical summaries, reshaping data. Learn: array creation, broadcasting, indexing/slicing, np.dot, np.random.

pandas

Pandas is a spreadsheet inside Python. Real-world data is messy. Pandas helps you load, clean, filter, group, and merge it. Learn: DataFrame, Series, read_csv, groupby, fillna, apply, merge, pivot_table.

matplotlib + seaborn

You cannot improve what you cannot see. Visualising data reveals patterns, outliers, and bugs. Learn: line plots, scatter plots, histograms, heatmaps, subplots. Seaborn makes pretty charts in 1 line.

2.3 Just-Enough Math for AI

You do NOT need a math degree. You need enough to read a blog post and not panic. Here is the minimum:

Vectors

Ordered lists of numbers $[x, y, z]$. A cat picture is a vector of 3072 numbers (32x32 pixels x 3 colours).

Matrices

Tables of numbers. Matrix multiplication = the engine under every neural network.

Derivatives

"If I nudge this dial, how much does the output change?" -- the core of learning.

Gradients

A list of derivatives for every dial. Gradient descent = turn all the dials to make the answer less wrong.

Probability

How likely is something? 90% confidence = the model is pretty sure. Essential for understanding predictions and uncertainty.

Resources for Math

3Blue1Brown -- Essence of Linear Algebra + Calculus (YouTube, ~4h total). Khan Academy -- probability & statistics (start at 'Basic probability'). StatQuest -- short, funny explanations of ML concepts.

2.4 Mini-Project: Linear Regression from Scratch

This is your first real ML project. Write it in pure Python + numpy, NO libraries. You will:

Generate fake data: $y = 2x + 1 + \text{some random noise}$

Write a prediction function: $y_{\text{pred}} = w * x + b$

Write a loss function: mean squared error

Compute gradients manually (or approximate them)

Write a gradient descent loop that updates w and b

Watch the loss go down and the line fit the data

Celebrate -- you just wrote a learning algorithm!

3. Months 4-6 -- Core Machine Learning

3.1 Classical ML with scikit-learn

Before deep learning, there was (and still is) 'classical' ML. These algorithms are simple, fast, and work well on tabular data (spreadsheets). Most business problems are solved with these, not neural networks.

- Linear / Logistic Regression -- predict a number or a yes/no
- Decision Trees & Random Forests -- the 'wisdom of the crowd' for tabular data
- SVM -- finds the best dividing line between two classes
- K-Nearest Neighbours -- look at the closest examples and vote
- K-Means clustering -- group similar things together with no labels
- PCA -- shrink 1000 columns down to 2 for visualisation

Learn the scikit-learn Pipeline API -- it chains preprocessing + model + prediction into one object. This is what production code looks like.

Critical concepts that apply everywhere: overfitting (memorising instead of learning), train/test split, cross-validation, bias-variance tradeoff, regularisation (L1/L2), evaluation metrics (accuracy, precision, recall, F1, ROC-AUC).

3.2 Deep Learning Foundations with PyTorch

PyTorch is the most popular deep learning framework. It does ONE magical thing: it tracks every calculation so it can automatically compute gradients (remember derivatives? autograd does them for you).

Build up step by step:

- [] Tensors -- numpy arrays that live on the GPU
- [] Autograd -- automatic gradient calculation
- [] nn.Module -- build your own neural network by stacking layers
- [] DataLoader -- load data in batches without writing a loop yourself
- [] Training loop -- forward pass, loss, backward(), optimizer.step()
- [] Move to GPU with .to('cuda') -- watch it go 10x faster

The 50-line Rule

A standard image classifier (CIFAR-10) is about 50 lines of PyTorch. If your code is longer than that, you are probably overcomplicating it.

3.3 MLOps Basics

MLOps = making ML work in the real world, not just on your laptop. Start with:

- Experiment tracking -- Weights & Biases or MLflow (log every run, compare results)
- Model serialisation -- torch.save / torch.load, ONNX export
- Simple serving -- FastAPI endpoint that loads a model and returns predictions
- Config files -- YAML configs so you never hardcode hyperparameters

3.4 Mini-Project: Cat vs Dog Classifier

Your first 'real' deep learning project. Use a dataset of 25,000 cat & dog images. You will learn about data loading, image transforms, convolutional layers, transfer learning (using a pretrained model), and avoiding overfitting.

4. Months 7-9 -- Specialise & Level Up

Now you pick a direction. All three tracks lead to a career, but they aim at different kinds of roles. Pick the one that excites you most.

4.1 TRACK A -- NLP / LLMs (Recommended)

Why this track?

LLMs are the hottest area in AI right now. Every company wants to build chatbots, summarisation tools, code assistants, and knowledge retrieval systems. Demand vastly outpaces supply.

Core topics to learn:

- Transformer architecture -- how 'Attention Is All You Need' works
- Tokenisation -- how text becomes numbers (and why it matters)
- Pretrained models -- downloading GPT, Llama, Mistral from Hugging Face
- RAG (Retrieval-Augmented Generation) -- chatbot that looks things up before speaking
- Vector databases -- Chroma, Pinecone, Qdrant -- store embeddings for fast search
- Fine-tuning -- LoRA / QLoRA -- adapt a big model cheaply
- Prompt engineering -- get better answers by asking better questions
- Agents -- LangGraph, CrewAI -- models that use tools and take actions
- Evaluation -- RAGAS, LLM-as-judge -- is your chatbot actually good?

Key Libraries

transformers (Hugging Face), langchain, llama-index, chromadb, vllm, ollama, pytorch

4.2 TRACK B -- Computer Vision

If you are fascinated by cameras, self-driving cars, medical imaging, or augmented reality, vision is your track.

- CNNs -- convolutions, pooling, strides, receptive fields
- Popular architectures -- ResNet, EfficientNet, YOLO
- Object detection -- finding and bounding-boxing objects in images
- Segmentation -- pixel-level classification (e.g. every pixel = road, car, sky)
- Image generation -- GANs, Diffusion models (Stable Diffusion)
- Data augmentation -- albumentations, torchvision transforms
- Video understanding -- optical flow, action recognition

Key Libraries

torchvision, detectron2, OpenCV, albumentations, supervision, fiftyone

4.3 TRACK C -- ML Platform / Infra

If you like DevOps, distributed systems, and making things run fast, this track makes you invaluable. You are the person who makes the data scientists' code actually work in production.

- Docker & Kubernetes -- containerise and orchestrate ML workloads
- Feature stores -- Feast, Tecton -- serve features consistently
- Model registries -- MLflow, BentoML -- version models like code
- CI/CD for ML -- DVC, CML, GitHub Actions for pipelines
- Distributed training -- Ray, DeepSpeed, FSDP -- train across multiple GPUs
- Monitoring -- Prometheus + Grafana, model drift detection
- Infrastructure as Code -- Terraform, Pulumi

5. Months 10-12 -- Production & Portfolio

5.1 Docker, CI/CD, Model Serving

A model in a Jupyter notebook is a prototype. A model in a Docker container behind an API is a product. Learn how to bridge the gap:

- Write a FastAPI app that loads your model and returns predictions via POST
- Write a Dockerfile that bundles your model + code + dependencies
- Add tests that run in CI (GitHub Actions)
- Deploy to a cloud VM or serverless (Railway, Modal, Hugging Face Spaces)
- Add a simple frontend (Gradio or Streamlit) so anyone can try it

5.2 Monitoring & Evaluation

Models degrade over time (data drift, concept drift). You need to know when:

- Log predictions and ground truth -- store them so you can compare
- Track accuracy / latency over time -- set up a dashboard
- Alerts -- if accuracy drops below 80%, notify someone
- Shadow / A/B testing -- compare your new model vs the old one on real traffic

5.3 Portfolio Projects That Stand Out

Your portfolio is more important than your resume. Build projects that show depth, not just completion of a tutorial. Aim for 3 projects:

Project 1: End-to-End RAG Chatbot

Ingest PDFs, embed them into a vector DB, answer questions with an LLM. Add evaluation (RAGAS scores), a Streamlit UI, deploy on Hugging Face Spaces. Show you understand the full pipeline from document to answer.

Project 2: Deep-Dive Implementation

Implement something from scratch to prove you understand the internals: a small transformer, a GAN that generates handwritten digits, or a custom training loop with mixed precision. A blog post explaining your implementation doubles the impact.

Project 3: Open Source Contribution

Fix a documentation bug, add a test, or implement a small feature in a library you use. This shows you can read other people's code and work in a team. Even a 5-line fix counts -- the goal is to experience the PR process.

6. Resources & Links

A curated list of the best free (or cheap) resources. Do not try to consume all of them. Pick one per topic and finish it.

Learn by Building

Fast.ai Practical Deep Learning -- teaches top-down: you build a working model in the first lesson, then learn the theory. Best for motivation.

CS Theory (the fun way)

Stanford CS229 (Andrew Ng) on YouTube. Old but gold. Also: Andrej Karpathy's Neural Networks: Zero to Hero playlist.

LLMs Specifically

Andrej Karpathy 'Let's build GPT' -- builds a mini GPT from scratch in 2 hours. Hugging Face NLP course -- practical, hands-on.

Production ML

Made With ML (madewithml.com) -- full-stack ML project. Full Stack Deep Learning course -- covers the entire lifecycle.

Stay Current

The Gradient newsletter. Arxiv abstracts (curated). r/MachineLearning on Reddit (filter by [D] discussion). Follow Karpathy, Jay Alammar, Lilian Weng on Twitter / blogs.

Practice & Compete

Kaggle -- datasets, competitions, notebooks. Do not spend too long on ranking; use it for structured practice.

Communities

PyTorch Discord, Hugging Face Discord, local AI meetups. Find study buddies -- learning alone is harder.

7. Weekly Action Plan

Consistency beats intensity. 10 hours per week for 52 weeks = 520 hours. That is enough to become employable. Here is a sample weekly schedule:

Weekday (Mon-Fri) 1 hour each evening

Watch a lecture OR code for 45 min + write notes for 15 min. Small daily wins compound.

Saturday 2-3 hours

Deep work: build something. No passive watching. Start a mini-project and push to GitHub.

Sunday 1 hour

Review the week. Read 1 blog post. Plan next week. Rest.

Monthly Milestones Checklist

- Month 1:** numpy + pandas basics done. Can load, filter, and plot a dataset.
- Month 2:** Wrote linear regression from scratch. Understands gradient descent.
- Month 3:** scikit-learn pipeline works on a real dataset. Knows train/test split.
- Month 4:** PyTorch basics: tensors, autograd, nn.Module. Trained a tiny network.
- Month 5:** Cat vs dog classifier works. Understands CNNs and transfer learning.
- Month 6:** FastAPI serving + experiment tracking set up. Can compare runs.
- Month 7:** Chosen a specialisation. Deep into it (LLMs / Vision / Infra).
- Month 8:** Built a project in your specialisation (RAG chatbot, object detector).
- Month 9:** Second project done. Starting to think about deployment.
- Month 10:** Docker + CI/CD pipeline working. Model served behind an API.
- Month 11:** All 3 portfolio projects solid. Blog posts written for 2 of them.
- Month 12:** Applied to jobs / freelance. One mock interview per week.

8. The Secret Sauce

Everyone says 'learn AI' but most people fail because they:

- Watch 100 hours of courses and never build anything --> tutorial hell.
- Try to learn ALL the math first --> burn out before writing a single line of ML code.
- Compare themselves to YouTube geniuses --> feel stupid and quit.

Your cheat code: build one dumb thing every 2 weeks. A price predictor. A cat detector. A 'does this email look like spam?' filter. Each one teaches you more than a textbook chapter.

Golden Rules

1. Code > Watch -- always. When you are tired, it is okay to watch a lecture. But when you have energy, code.
2. Ship early -- a bad project on GitHub is worth more than a perfect one in your head.
3. Teach to learn -- write blog posts, make videos, explain to friends. Teaching reveals gaps in your understanding.
4. AI is a tool, not magic -- know when to use it and when not to. A simple rule-based system often beats a complex model.
5. You will feel stupid -- every single day. That is how learning feels. Keep going.

"The best time to start was yesterday.

The second best time is now."

Good luck. You have got this!